

千问AI视频前端研发

一面（技术基础） · 1-10

1. 请简述 CSS 中的 Flex 布局及其常用属性。
2. 谈谈你对 JavaScript 闭包的理解。
3. Promise 有哪几种状态？async/await 和它是什么关系？
4. 什么是 SSE（Server-Sent Events）？它在 AI 对话场景中有什么优势？
5. 如何使用 Canvas 绘制一个简单的视频进度条预览图？
6. 手写代码：实现一个基础的防抖函数（debounce）。
7. 数组中有多个对象，如何根据对象的某个属性进行去重？
8. 介绍一下浏览器的事件循环（Event Loop）机制。
9. CSS 中实现水平垂直居中的方式有哪些？
10. 算法题：给定一个字符串，判断其是否为有效的括号组合。

二面（深入原理与项目） · 1-12

1. React 的 Fiber 架构解决了什么问题？它的工作原理是什么？
2. 在 AI 对话产品中，如何实现 Markdown 内容的实时流式渲染？
3. 如何利用 Web Workers 处理视频帧数据以避免主线程卡顿？
4. 谈谈大文件上传（如4K视频）的整体流程设计。
5. 在 AI 视频生成场景中，前端如何管理复杂的 Prompt 模板与参数？
6. 你们项目里是如何做性能优化的？请列举具体的指标和手段。
7. WebCodecs API 了解吗？它与传统的 video 标签处理有什么区别？
8. 谈谈前端安全，特别是针对 AI 产品的 Prompt Injection（提示词注入）防范。
9. Vite 为什么比 Webpack 快？它的构建原理是什么？
10. 状态管理库（如 Zustand 或 Redux）在复杂 AI 工作流中如何设计？
11. 手写代码：实现一个简单的 Promise.all。
12. 如何处理AI接口响应时间过长导致的超时问题？

三面（架构与综合能力） 1-8

1. 如果让你从零设计一个 AI 视频编辑器前端架构，你会考虑哪些核心模块？

2. 前端如何利用 AI 提升研发效率？谈谈你的实践或思考。
3. 在 AI 视频产品中，如何解决模型生成结果不确定性带来的 UI 挑战？
4. 聊聊你在项目中遇到的最难的技术挑战，你是如何解决的？
5. 如何设计一个通用的 AI Agent 配置化组件？
6. 谈谈你对 WebGPU 的看法，它会对 AI 视频前端开发带来什么改变？
7. 如果项目需要支持多语言（国际化），在 AI 视频场景下有哪些特殊考虑？
8. 你未来的职业规划是什么？在 AI 领域有什么持续学习的计划？

一面（技术基础）

1. 请简述 CSS 中的 Flex 布局及其常用属性。

难度：★

考点：CSS 布局基础

答案要点：

Flex 布局即弹性布局，旨在为盒状模型提供最大的灵活性，通过设置容器的 display 属性为 flex 来开启。

- 容器属性：flex-direction 决定主轴方向，justify-content 处理主轴对齐，align-items 处理交叉轴对齐。
- 项目属性：flex-grow 定义放大比例，flex-shrink 定义缩小比例，flex-basis 定义初始空间。
- 响应式优势：能够轻松实现垂直居中以及等高布局，适应不同屏幕尺寸。
- 渲染性能：相比传统的浮动布局，Flex 布局在现代浏览器中的重排重绘效率更高。

常见坑：

- 忽略 flex-shrink 的默认值为 1，导致内容被意外挤压。
- 在 flex 容器中使用 margin: auto 实现居中的原理理解不深。

追问：

- flex: 1 是哪三个属性的简写？

2. 谈谈你对 JavaScript 闭包的理解。

难度：★

考点：JS 作用域与内存

答案要点：

闭包是指有权访问另一个函数作用域中变量的函数，本质是函数与其词法环境的引用捆绑在一起。

- 形成条件：函数嵌套且内部函数引用了外部函数的变量。
- 核心作用：实现数据的私有化，防止全局变量污染，常用于模块化封装。
- 内存影响：由于内部函数持有外部变量引用，会导致相关变量无法被垃圾回收机制自动回收。

- 实际应用：在高阶函数、柯里化、防抖节流函数中广泛使用。

常见坑：

- 在循环中创建闭包未注意变量捕获时机。
- 滥用闭包导致内存泄漏。

追问：

- 闭包会导致内存泄漏吗？如何手动解除引用？

3. Promise 有哪几种状态？async/await 和它是什么关系？

难度：★★

考点：异步编程

答案要点：

Promise 是异步编程的一种解决方案，包含 Pending、Fulfilled 和 Rejected 三种状态，状态一旦改变不可逆。

- 状态流转：从 Pending 变为成功态或失败态，通过 resolve 和 reject 函数触发。
- 链式调用：then 方法返回一个新的 Promise，解决了回调地狱问题。
- 语法糖关系：async/await 是 Promise 的语法糖，使异步代码在书写上更接近同步逻辑。
- 错误处理：async/await 配合 try-catch 使用，比 Promise.catch 更符合直觉。

常见坑：

- 忘记在 async 函数中 await 异步操作。
- 在 forEach 循环中使用 await 导致并发逻辑失效。

追问：

- Promise.all 和 Promise.allSettled 有什么区别？

4. 什么是 SSE (Server-Sent Events)? 它在 AI 对话场景中有什么优势？

难度：★★

考点：网络协议与 AI 应用

答案要点：

SSE 是一种基于 HTTP 的服务器向浏览器推送实时数据的技术，允许服务器单向发送文本流。

- 协议特点：基于标准 HTTP 协议，支持断线重连，比 WebSocket 更轻量。
- AI 场景适配：LLM 模型通常是流式输出，SSE 能够实现打字机效果，提升用户感知速度。
- 实现方式：设置 Content-Type 为 text/event-stream，通过事件流持续传输数据。
- 兼容性：现代浏览器原生支持 EventSource API，简单易用。

常见坑：

- 忽略了 HTTP/1.1 下浏览器对同一域名的并发连接数限制（通常为 6 个）。

- 未正确处理数据传输结束的标志位。

追问：

- SSE 和 WebSocket 在双向交互上有什么区别？

5. 如何使用 Canvas 绘制一个简单的视频进度条预览图？

难度：★★

考点：Canvas API 与视频处理

答案要点：

通过 Canvas 的 `drawImage` 方法可以捕获视频帧并将其绘制到画布上，从而生成预览图。

- 核心步骤：获取 video 元素，设置 canvas 宽高，调用 `ctx.drawImage(video, 0, 0)`。
- 时间点控制：通过修改 `video.currentTime` 跳转到指定时间，监听 `seeked` 事件后再绘制。
- 性能优化：对于大量预览图，可以使用离屏 Canvas 进行预渲染。
- 导出格式：使用 `canvas.toDataURL` 或 `toBlob` 将绘制结果转换为图片资源。

常见坑：

- 视频跨域导致 Canvas 被污染，无法导出图片。
- 在视频数据未加载完成时尝试绘制导致黑屏。

追问：

- 如果要实现视频抽帧，如何保证抽帧的效率？

6. 手写代码：实现一个基础的防抖函数 (debounce)。

难度：★★

考点：手写代码能力

答案要点：

防抖是指在事件被触发 n 秒后再执行回调，如果在这 n 秒内又被触发，则重新计时。

- 核心逻辑：利用闭包存储 timer 变量。
- 清除机制：每次触发时先执行 `clearTimeout`。
- 延迟执行：使用 `setTimeout` 包装目标函数。
- 参数传递：确保 `this` 指向正确并透传 arguments。

代码示例：

代码块

```
1 function debounce(fn, delay) {
2   let timer = null;
3   return function(...args) {
4     if (timer) clearTimeout(timer);
5     timer = setTimeout(() => {
6       fn.apply(this, args);
```

```
7     }, delay);  
8   };  
9 }
```

常见坑：

- 丢失了原函数的 this 指向。
- 忘记在闭包中返回函数。

7. 数组中有多个对象，如何根据对象的某个属性进行去重？

难度：★

考点：数据处理

答案要点：

可以使用 Map 或 Set 结合 filter 方法，利用键值的唯一性进行过滤。

- Map 方案：遍历数组，以属性值为 key 存入 Map，判断 key 是否存在。
- 性能优势：Map 的查找复杂度为 $O(1)$ ，适合处理大数据量。
- 纯函数：不修改原数组，返回一个新数组。
- 灵活性：可以轻松扩展为根据多个属性组合去重。

代码示例：

代码块

```
1  const unique = (arr, key) => {  
2    const map = new Map();  
3    return arr.filter(item => !map.has(item[key]) && map.set(item[key], true));  
4  };
```

常见坑：

- 使用双重 for 循环导致 $O(n^2)$ 的复杂度。
- 忽略了属性值为 undefined 或 null 的边界情况。

8. 介绍一下浏览器的事件循环 (Event Loop) 机制。

难度：★★

考点：浏览器原理

答案要点：

事件循环是 JavaScript 处理异步任务的核心机制，通过任务队列协调主线程的执行顺序。

- 任务分类：分为宏任务（setTimeout, script）和微任务（Promise.then, MutationObserver）。
- 执行顺序：执行一个宏任务 -> 清空所有微任务 -> 渲染页面 -> 执行下一个宏任务。

- 渲染时机：微任务清空后、宏任务执行前是浏览器尝试更新 UI 的时机。
- 阻塞后果：主线程长任务会阻塞事件循环，导致页面卡顿。

常见坑：

- 认为 `setTimeout(fn, 0)` 会立即执行。
- 混淆了 Node.js 和浏览器的事件循环差异。

9. CSS 中实现水平垂直居中的方式有哪些？

难度：★

考点：CSS 基础

答案要点：

实现居中的方式多种多样，应根据具体布局场景选择最简洁的方案。

- Flex 方案：`justify-content: center + align-items: center`。
- Grid 方案：`display: grid + place-items: center`。
- 绝对定位：`top: 50% + left: 50% + transform: translate(-50%, -50%)`。
- Margin 方案：对于已知宽高的元素，使用 `margin: auto`。

常见坑：

- 使用 `transform` 居中可能导致文字模糊（像素对齐问题）。
- 绝对定位脱离文档流对周围元素的影响。

10. 算法题：给定一个字符串，判断其是否为有效的括号组合。

难度：★★

考点：栈结构应用

答案要点：

利用栈（Stack）先进后出的特性，遍历字符串进行匹配。

- 入栈逻辑：遇到左括号（`(`、`[`、`{`）则压入栈中。
- 出栈逻辑：遇到右括号，判断栈顶元素是否匹配，不匹配或栈为空则返回 `false`。
- 最终检查：遍历结束后，如果栈为空则说明全部匹配成功。
- 复杂度：时间复杂度 $O(n)$ ，空间复杂度 $O(n)$ 。

常见坑：

- 忘记处理字符串为空的情况。
- 遍历结束后栈内仍有剩余左括号的情况。

二面（深入原理与项目）

1. React 的 Fiber 架构解决了什么问题？它的工作原理是什么？

难度：★★★

考点：框架深度原理

答案要点：

Fiber 架构主要解决了 React 在处理复杂组件树时，长任务阻塞主线程导致页面卡顿的问题。

- 核心目标：实现增量渲染，将渲染工作拆分为多个小任务，支持任务的优先级和中断。
- 数据结构：Fiber 是一个链表结构，记录了组件的类型、DOM 节点及兄弟/子节点引用。
- 两个阶段：Render 阶段（可中断，计算差异）和 Commit 阶段（不可中断，同步更新 DOM）。
- 调度器：通过 Scheduler 调度任务，利用浏览器空闲时间（requestIdleCallback 思想）执行。

常见坑：

- 误以为 Fiber 是为了提高渲染绝对速度（实际上是为了提高响应性）。
- 在 Render 阶段执行具有副作用的操作。

追问：

- 为什么 React 17+ 之后不再使用 requestIdleCallback？

2. 在 AI 对话产品中，如何实现 Markdown 内容的实时流式渲染？

难度：★★★

考点：AI 交互优化

答案要点：

流式渲染需要处理不完整的 Markdown 片段，确保在数据持续到达时 UI 不闪烁且解析正确。

- 解析策略：使用支持流式的 Markdown 解析器（如 markdown-it），对增量字符串进行实时解析。
- 性能优化：避免全量重新渲染，通过 diff 算法或仅渲染新增的 token 块。
- 代码高亮：配合 prismjs 或 highlight.js，需处理代码块未闭合时的展示逻辑。
- 自动滚动：监听内容高度变化，在用户未手动干预时保持滚动条触底。

常见坑：

- 解析器在处理截断的 HTML 标签或 Markdown 语法（如表格）时崩溃。
- 频繁的 DOM 操作导致低端设备 CPU 占用过高。

追问：

- 如何处理流式输出过程中的公式（MathJax/KaTeX）渲染？

3. 如何利用 Web Workers 处理视频帧数据以避免主线程卡顿？

难度：★★★

考点：前端性能与多线程

答案要点：

Web Workers 允许在后台线程执行耗时计算，非常适合处理视频解码、滤镜计算等任务。

- 数据传输：使用 Transferable Objects（如 ArrayBuffer）转移所有权，避免高昂的克隆成本。
- 离屏渲染：利用 OffscreenCanvas 在 Worker 中直接进行图形绘制。
- 通信机制：通过 postMessage 交换控制指令和处理后的帧数据。
- 场景应用：视频实时水印、人脸检测算法、视频转码预处理。

常见坑：

- 在 Worker 中尝试操作 DOM 导致报错。
- 频繁的 postMessage 通信开销抵消了多线程带来的收益。

追问：

- 如何在 Worker 中加载第三方处理库？

4. 谈谈大文件上传（如 4K 视频）的整体流程设计。

难度：★★

考点：工程实践

答案要点：

大文件上传的核心是分片上传、断点续传和秒传，以提高成功率和用户体验。

- 分片逻辑：使用 Blob.slice 将文件切分为固定大小的 Chunk。
- 并发控制：控制同时上传的分片数量，避免占用过多 TCP 连接。
- 断点续传：上传前先调用接口查询已上传的分片索引，仅上传缺失部分。
- 秒传实现：计算文件完整 MD5 值，若服务器已存在该文件则直接返回成功。

常见坑：

- MD5 计算大文件时阻塞主线程（应使用 Worker 或抽样 Hash）。
- 最后一个分片上传完成后，合并请求的超时处理。

追问：

- 如果某个分片上传失败，你的重试机制是怎么设计的？

5. 在 AI 视频生成场景中，前端如何管理复杂的 Prompt 模板与参数？

难度：★★

考点：AI 组件设计

答案要点：

Prompt 管理需要兼顾灵活性和结构化，通常采用配置驱动的模式。

- 模板化：使用占位符（如 {{subject}}）定义 Prompt 模板，动态注入用户输入。
- 结构化存储：将 Prompt 拆分为前缀、主体、风格修饰词、负向提示词等字段。

- 版本控制：前端配合后端记录 Prompt 的版本，支持用户回溯生成效果。
- 交互设计：提供可视化标签（Tags）选择，降低用户的输入门槛。

常见坑：

- 直接拼接字符串导致特殊字符转义错误。
- 忽略了不同模型对 Prompt 长度限制的差异。

追问：

- 如何实现 Prompt 的自动补全或智能建议？

6. 你们项目里是如何做性能优化的？请列举具体的指标和手段。

难度：★★

考点：性能优化

答案要点：

性能优化应以数据为导向，关注核心 Web 指标（LCP, FID, CLS）。

- 加载优化：路由懒加载、组件异步导入、CDN 加速、Gzip/Brotli 压缩。
- 渲染优化：减少重排重绘、使用 RequestAnimationFrame、虚拟列表处理长列表。
- AI 场景专项：预加载模型文件、流式接口首字节优化（TTFB）。
- 监控方案：接入性能监控平台，收集真实用户（RUM）数据。

常见坑：

- 过度优化：引入了复杂的缓存逻辑反而增加了维护成本。
- 只关注首屏，忽略了交互后的内存泄露。

7. WebCodecs API 了解吗？它与传统的 video 标签处理有什么区别？

难度：★★★

考点：视频底层技术

答案要点：

WebCodecs 提供了对浏览器内置编解码器的底层访问权限，允许开发者直接处理原始视频帧。

- 核心能力：直接操作 VideoFrame 和 AudioData，支持自定义封装格式。
- 性能优势：相比于 ffmpeg.wasm（软件解码），WebCodecs 使用硬件加速，效率极高。
- 灵活度：可以实现精细的视频编辑、实时滤镜叠加、低延迟直播流处理。
- 配合 API：常与 WebGPU 或 WebGL 结合进行高性能图像处理。

常见坑：

- 兼容性问题：目前仅在 Chromium 系浏览器支持较好。
- 内存管理：VideoFrame 使用完后必须手动调用 close() 释放，否则会导致内存溢出。

追问：

- 为什么在处理视频编辑时，WebCodecs 比 Canvas 捕获更优？

8. 谈谈前端安全，特别是针对 AI 产品的 Prompt Injection（提示词注入）防范。

难度：★★

考点：安全意识

答案要点：

虽然 Prompt Injection 主要在后端防御，但前端在输入过滤和展示安全上仍有职责。

- 输入限制：对用户输入的长度、特殊字符进行校验，防止恶意构造指令。
- 内容安全策略 (CSP)：严格配置 CSP，防止 AI 输出的恶意脚本被执行。
- 渲染转义：确保 AI 返回的内容在渲染为 HTML 前经过严格转义，防止 XSS。
- 敏感信息脱敏：在发送 Prompt 前，前端可进行初步的正则匹配，提醒用户不要上传密钥等信息。

常见坑：

- 盲目信任 AI 输出的内容，直接使用 `v-html` 或 `dangerouslySetInnerHTML`。
- 忽略了 URL 参数中可能携带的注入攻击。

9. Vite 为什么比 Webpack 快？它的构建原理是什么？

难度：★★

考点：工程化工具

答案要点：

Vite 在开发环境下利用了浏览器原生的 ES Modules (ESM) 特性，跳过了打包过程。

- 开发环境：按需编译，只有当浏览器请求某个模块时，Vite 才会处理该模块。
- 依赖预构建：使用 esbuild 对第三方库进行预打包，esbuild 由 Go 编写，速度极快。
- 生产环境：使用 Rollup 进行打包，保证了构建产物的优化。
- 热更新 (HMR)：由于模块间解耦，HMR 的速度不随项目体积增大而变慢。

常见坑：

- 某些老旧库不支持 ESM，导致 Vite 开发环境下报错。
- 开发环境与生产环境行为不一致（ESM vs Rollup）。

10. 状态管理库（如 Zustand 或 Redux）在复杂 AI 工作流中如何设计？

难度：★★

考点：架构设计

答案要点：

在 AI 视频生成等复杂流程中，状态管理需要处理大量的异步状态和中间过程。

- 状态拆分：将 UI 状态（弹窗、加载中）与业务数据（视频列表、模型参数）分离。
- 异步流处理：利用中间件或内置机制处理长耗时的 AI 生成任务状态轮询。

- 响应式设计：确保 Prompt 变更能实时同步到预览组件。
- 持久化：对用户配置的复杂参数进行本地存储，防止刷新丢失。

常见坑：

- 状态结构嵌套过深，导致更新逻辑复杂且易出错。
- 频繁更新全局状态导致无关组件大量重渲染。

11. 手写代码：实现一个简单的 Promise.all。

难度：★★

考点：手写代码能力

答案要点：

Promise.all 接收一个 Promise 数组，返回一个新的 Promise，要求所有任务都成功才成功。

- 计数器：维护一个 count 变量记录已完成的任务数。
- 结果存储：创建一个结果数组，按输入顺序存放 resolve 的值。
- 失败处理：只要有一个任务 reject，整体立即 reject。
- 边界处理：处理输入为空数组的情况。

代码示例：

代码块

```
1  Promise.myAll = function(promises) {
2    return new Promise((resolve, reject) => {
3      let count = 0;
4      const results = [];
5      if (promises.length === 0) return resolve([]);
6      promises.forEach((p, i) => {
7        Promise.resolve(p).then(res => {
8          results[i] = res;
9          count++;
10         if (count === promises.length) resolve(results);
11       }, reject);
12     });
13   });
14 }
```

12. 如何处理 AI 接口响应时间过长导致的超时问题？

难度：★★

考点：异常处理与用户体验

答案要点：

处理长耗时请求需要从技术容错和用户感知两个维度入手。

- 技术手段：设置合理的 timeout，使用 AbortController 实现请求主动截断。
- 轮询机制：对于超长任务，前端发起任务后，通过短轮询或 WebSocket 监听进度。
- 体验优化：展示进度条、预计剩余时间，或提供“后台生成”选项。
- 容错重连：接口报错时实施指数退避算法的自动重试。

常见坑：

- 超时时间设置过短，导致大模型还没计算完就被前端切断。
- 界面无反馈，让用户以为页面挂掉了。

三面（架构与综合能力）

1. 如果让你从零设计一个 AI 视频编辑器前端架构，你会考虑哪些核心模块？

难度：★★★

考点：系统设计

答案要点：

一个成熟的 AI 视频编辑器需要处理轨道管理、实时预览、AI 算子集成等核心功能。

- 渲染引擎层：基于 Canvas/WebGL 或 WebCodecs 的视频渲染与特效合成模块。
- 状态管理层：处理多轨道 Timeline、关键帧数据、素材库状态的复杂 Store。
- AI 服务层：封装 Prompt 调用、视频风格化、自动字幕生成等 AI 能力的 SDK。
- 资源调度层：负责大文件分片上传、素材预加载、离屏渲染任务分配。
- 交互层：高度可定制的 Timeline 组件、属性编辑面板、实时预览窗口。

常见坑：

- 忽略了撤销重做（Undo/Redo）在复杂编辑器中的实现难度。
- 轨道数据与 DOM 强耦合，导致性能瓶颈。

2. 前端如何利用 AI 提升研发效率？谈谈你的实践或思考。

难度：★★

考点：技术视野

答案要点：

AI 对前端研发的提升体现在代码编写、测试、文档及 UI 转换等多个环节。

- 代码辅助：利用 Copilot/Cursor 进行逻辑实现、单元测试自动生成。
- UI 自动化：尝试 Design-to-Code 工具，将 Figma 稿直接转换为 React/Vue 组件。
- 调试增强：利用 LLM 辅助分析复杂的错误堆栈或性能瓶颈。

- 流程优化：编写自定义脚本，调用 AI 接口自动翻译多语言文件或生成组件文档。

常见坑：

- 过度依赖 AI 生成的代码，忽略了代码质量审核和安全隐患。
- 无法准确描述需求（Prompt 能力不足），导致 AI 输出结果不可用。

3. 在 AI 视频产品中，如何解决模型生成结果不确定性带来的 UI 挑战？

难度：★★★

考点：产品思维与技术结合

答案要点：

AI 生成具有随机性，前端需要通过设计手段引导用户并处理异常结果。

- 结果预览：提供多版本并行生成预览，让用户进行对比选择（A/B 选优）。
- 参数约束：通过前端表单校验和滑块限制，将模型参数控制在易出好结果的范围内。
- 状态反馈：细化生成步骤（如：理解需求中、绘制草图、渲染视频），缓解用户焦虑。
- 兜底策略：当生成失败或内容违规时，提供清晰的错误提示及重试建议。

常见坑：

- 界面只显示一个 Loading，用户不知道当前 AI 处理到了哪一步。
- 未处理 AI 返回的非法格式数据导致页面白屏。

4. 聊聊你在项目中遇到的最难的技术挑战，你是如何解决的？

难度：★★★

考点：问题解决能力

答案要点：

此题重点在于体现解决问题的思路、使用的工具以及最终带来的业务价值。

- 背景描述：清晰介绍当时的技术背景、业务压力和核心难点（如：视频合成卡顿）。
- 排查过程：使用了哪些工具（Chrome DevTools, Performance Profiler, 抓包等）。
- 方案对比：当时考虑了哪几种方案，为什么最终选择了这一种（权衡利弊）。
- 最终结果：量化的性能提升数据或业务指标增长，以及后续的沉淀总结。

常见坑：

- 描述过于琐碎，没有重点。
- 解决方案听起来太简单，体现不出“挑战性”。

5. 如何设计一个通用的 AI Agent 配置化组件？

难度：★★★

考点：组件抽象能力

答案要点：

Agent 组件需要具备高度的可扩展性，以适配不同能力的模型和工具调用。

- 协议标准化：定义统一的 JSON Schema 来描述 Agent 的能力、输入输出参数。
- 动态表单：根据 Schema 自动生成配置界面，支持插件式的工具挂载。
- 交互解耦：将对话展示逻辑与底层的模型调用逻辑分离，支持更换不同的 LLM 供应商。
- 状态追踪：可视化展示 Agent 的思考过程（Thought）和执行动作（Action）。

常见坑：

- 组件与特定后端接口强绑定，无法复用。
- 忽略了复杂嵌套参数的校验逻辑。

6. 谈谈你对 WebGPU 的看法，它会对 AI 视频前端开发带来什么改变？

难度：★★★

考点：前沿技术

答案要点：

WebGPU 是下一代 Web 图形标准，提供了比 WebGL 更直接的 GPU 访问能力。

- 计算能力：支持 Compute Shader，使得在前端进行大规模并行计算（如视频转码、AI 推理）成为可能。
- 性能飞跃：减少了 CPU 到 GPU 的通信开销，渲染复杂视频特效时帧率更稳定。
- 端侧 AI：配合 Transformers.js 等库，可以在浏览器端直接运行小型模型，减少服务器压力。
- 行业趋势：是未来高性能 Web 应用（编辑器、游戏、AI 工具）的核心基石。

常见坑：

- 忽略了目前 WebGPU 尚不完美的兼容性。
- 学习曲线陡峭，对开发者的图形学基础要求较高。

7. 如果项目需要支持多语言（国际化），在 AI 视频场景下有哪些特殊考虑？

难度：★★

考点：工程化周全性

答案要点：

除了基础的文本翻译，AI 视频产品还涉及音视频同步和文化差异。

- 语音对齐：AI 生成的配音在不同语言下长度不同，前端需动态调整视频帧或字幕时间轴。
- 字体支持：视频渲染引擎需支持多国语言字体，防止出现方块字。
- 布局自适应：处理 RTL（从右向左）语言对 UI 布局的影响。
- 敏感词库：不同国家对 AI 生成内容的合规性要求不同，前端需配合做差异化过滤。

常见坑：

- 硬编码了视频中的文字，导致无法翻译。

- 忽略了数字、日期格式在不同国家的展示习惯。

8. 你未来的职业规划是什么？在 AI 领域有什么持续学习的计划？

难度：★

考点：稳定性与自驱力

答案要点：

面试官希望看到候选人有明确的目标感和对技术的热情。

- 深度与广度：在精通前端架构的基础上，深入学习 AI 相关的底层原理（如 Transformer 结构）。
- 业务结合：探索如何将最新的 AI 研究成果转化为可落地的用户产品。
- 社区参与：关注开源项目，参与技术分享，保持对行业动态的敏感度。
- 学习路径：例如阅读相关论文、实践端侧模型部署、掌握 Python/LangChain 等跨界技能。

常见坑：

- 回答过于笼统（如“努力工作”）。
- 表现出对前端基础的厌倦，只追求 AI 热点。